

Development of a Multiplayer XR Drone Game in Unity Using ZED Camera

Yunhan Wei

University of California, Davis

Spring 2026

Abstract

This project proposes and implements an engineering solution for “developing a Unity drone game with the ZED Camera,” with the goal of integrating spatial localization optimization and multiplayer networking into a single runnable game. The system uses ZED camera poses as its primary input, performs spatial alignment and visual correction on the Unity side, and focuses on resolving the misalignment between virtual objects and the real scene as well as scale bias. On this basis, it provides a multiplayer scheme that allows different cameras to interact under a unified game-space reference [1, 2].

For spatial localization optimization, the system first corrects the proportional error between the virtual world and reality, then uses multi-anchor fitting to simultaneously correct overall orientation and positional bias, and continuously performs small adaptive refinements at runtime to reduce drift. The whole calibration is split into two parts: it strictly separates the “visual correction” from the “game logic coordinates,” so that the local calibration process does not interfere with multiplayer synchronization or gameplay decisions.

For multiplayer networking, the system handles same-room connection and key network object synchronization, and the host broadcasts a unified calibration snapshot in a single shot, so that all clients start the game under the same spatial reference, significantly reducing scene misalignment across viewpoints.

1 Introduction and Background

The lab’s research focuses on developing a multiplayer XR drone system, enabling multiple drones to coexist and interact in real time within shared physical and virtual spaces. Ideally, this system integrates drone hardware, cameras, external localization, and the game engine to achieve localization, control, and gameplay within a unified coordinate system.

When I joined the lab, the physical drones were still under repair and integration, and the complete hardware pipeline was not yet reliable for development verification. Waiting until the hardware was fully restored before advancing the software side would have significantly delayed validation of the game itself. Given this practical constraint, this project chose a feasible transitional strategy: relying solely on the ZED Camera as the primary spatial input, and getting a runnable game.

This project does not attempt spatial memory or mapping, instead, it focuses on reliably using position data generated by ZED camera. Common errors in XR games include scale bias, orientation bias, overall translation bias, and runtime drift. If these errors are left unhandled,

players will see clearly visible misalignment between the virtual and real worlds, and in multiplayer sessions, different clients will end up with inconsistent world references. To address this, the project implements a multi-stage calibration pipeline and explicitly separates the local visual corrections produced by calibration from the world reference shared across players, in order to simultaneously improve spatial alignment stability and reduce the practical impact of drift [2, 3, 9].

In addition to spatial calibration, the project feeds the calibration results into the multiplayer flow. Through Photon PUN’s room connection, network object synchronization, and calibration snapshot broadcasting, the host and clients can start the game under a unified reference, avoiding situations where “the same GameObject ends up at different positions on different devices.” Finally, the project integrates these capabilities into a playable demo, forming a complete validation loop from calibration to networking to combat cycle.

1.1 Prior Work

The lab already had hardware and development setup for XR drone projects, and previous students had explored several directions for connecting drone hardware, external sensing devices, and Unity-based AR visualization. However, getting all of the hardware components to work together was difficult, and debugging the complete system was both complicated and time-consuming. In addition, the scripts left by previous students were written for different versions of Unity and depended on different software packages, so directly reusing them would likely have introduced compatibility problems and made long-term maintenance harder.

There was also an important historical reason behind the hardware choices in the earlier stages of the project. At that time, the SLAM SDK was not available for the ZED Camera used for the AR video feed, so the lab purchased separate RealSense cameras to provide localization and spatial tracking support. However, the RealSense-based interface introduced its own integration problems, and the additional hardware also increased the complexity of the system. Since the ZED Camera SDK now provides usable spatial tracking and localization support, this project revisits the system design and simplifies the pipeline by relying primarily on the ZED Camera as the main spatial input source.

Against this background, using only the ZED Camera to build the game became a more executable and iterative baseline. This approach makes it possible to validate the key pipeline from spatial localization to gameplay logic without depending on the full drone hardware loop. Building on the prior exploration of the ZED Camera, I designed and implemented a game-side solution that uses data from the ZED Camera SDK for spatial alignment and gameplay driving. The system was then tested and tuned against common problems such as scale mismatch, unstable alignment, and positional drift, bringing the overall game performance to a reasonably stable state.

In earlier approaches, multiplayer mainly worked by sending data from the drones to Unity and allowing two drones to share the same data source. In this project, instead of using the drone as the main input source, I connected the ZED Camera directly to Unity. This shift also changed the focus of the multiplayer system: rather than simply sending data in one direction, the system now needs to ensure that multiple ZED clients remain synchronized under the same world reference. To achieve this, I built on the old data transmission idea and added a host-broadcast mechanism, where the host sends calibration information to all clients in the room simultaneously. This helps all clients start from the same spatial reference and keeps spatial alignment more consistent during multiplayer sessions.

1.2 Contributions

The main contributions of this project are as follows:

1. Implemented a scale alignment tool that corrects the proportional bias between virtual content and the real movement distance.
2. Implemented a spatial recognition and global alignment optimization process that uses multiple anchors and continuous online adjustments to accurately correct orientation, position, and runtime drift.
3. Implemented multiplayer synchronization features, including room connection, network object synchronization, and calibration data broadcasting, which improve the consistency of the spatial reference across clients.
4. Formed a playable demo loop: integrated the calibration module, network synchronization, and game, validating the usability of the above solution in actual gameplay.

1.3 System Design

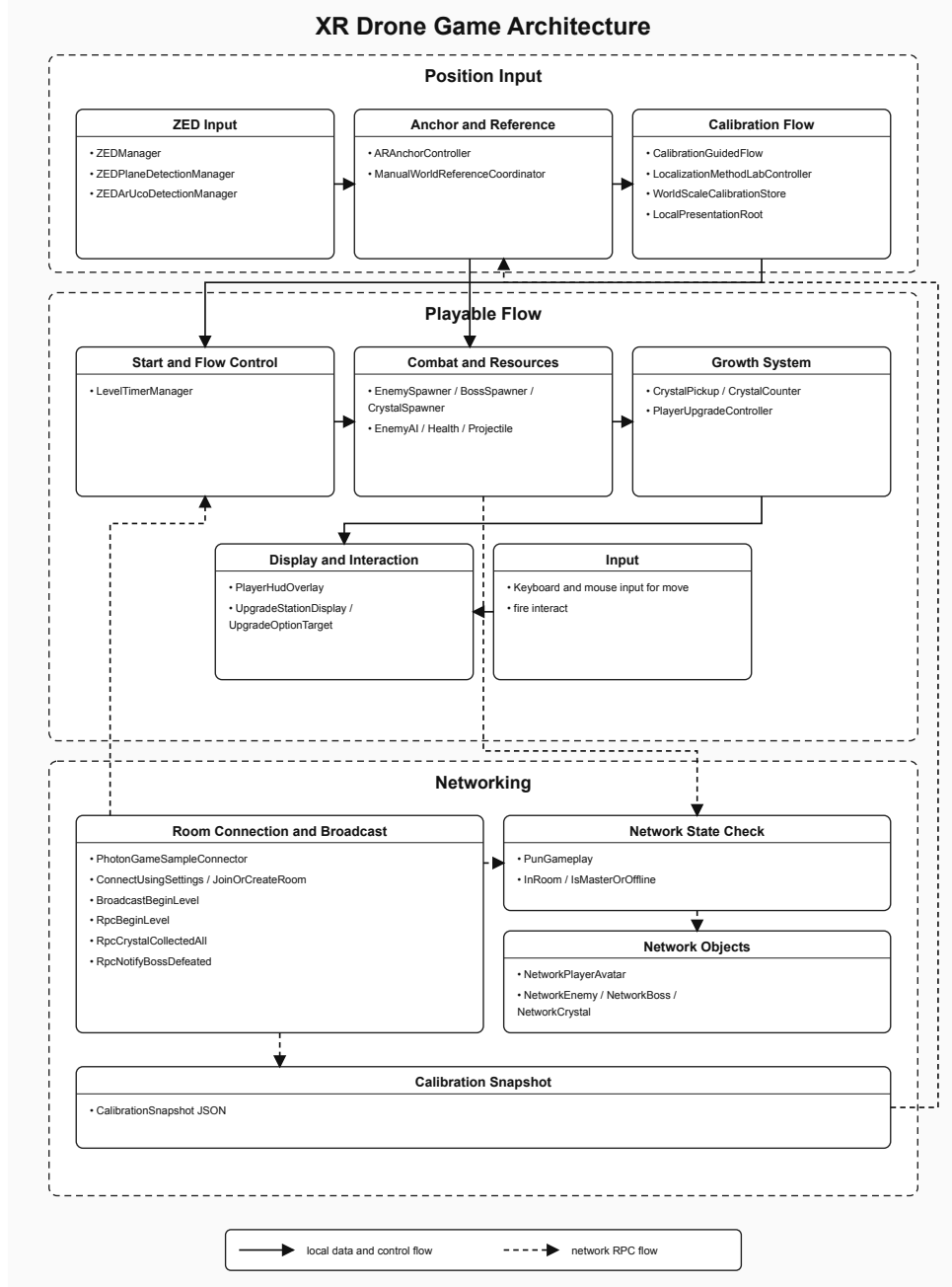


Figure 1: System Architecture of the Multiplayer XR Drone Game

2 Implementation

The implementation of this project can be divided into two parts: spatial and multiplayer.

Its goal is to convert the ZED camera’s local tracking data into a stable coordinate system that the game can use. This includes: first performing scale alignment and anchoring the virtual world to the real floor (see 2.1); then using a multi-anchor robust similarity transform to correct scale, yaw, and translation in a single shot (see 2.2); and performing small periodic adaptive refinements at runtime to mitigate drift over long sessions (see 2.3) [3, 4, 7, 8, 9].

The second part addresses the multiplayer side. It lets the two players share the same world

reference. The approach is that, after the host completes its local spatial calibration, it broadcasts the key world reference once to all clients in the room, so that both ends start the game on the same spatial baseline (see 2.4).

The use of broadcasting means that the host directly transmits its own data, which makes the client highly dependent on the host’s spatial localization capability. That’s why both parts—the spatial alignment and network synchronization—are crucial to the system. Section 2.5 then discusses the limitations that remain in the current implementation.

2.1 Spatial Calibration on Top of ZED

In this project, on top of the local tracking results that ZED already provides, I add two constraints that are most critical to the XR experience: “real-world scale” and “anchoring” [2, 4].

2.1.1 Initial Scale and Yaw Alignment

The first issue to address is the initial consistency of scale and orientation. Although the poses provided by ZED can continuously describe camera motion, in any given session the displacement length it reports may have a scale bias relative to real metric units. At the same time, the direction the player happens to face at startup is essentially arbitrary; if this orientation difference is not first reconciled, the same logical level will be projected into the real world at a different yaw in different sessions.

The project uses a very direct but effective approach for the initial calibration: the player walks an approximately straight line of known length between two points; the system records the displacement length L_{zed} reported by ZED, while the player manually enters the real distance L_{real} measured with a tape measure. From this we obtain a unified scale correction factor [1, 2]

$$s = \frac{L_{\text{real}}}{L_{\text{zed}}} = \frac{\|\mathbf{B}_{\text{real}} - \mathbf{A}_{\text{real}}\|}{\|\mathbf{B}_{\text{zed}} - \mathbf{A}_{\text{zed}}\|}$$

In addition to the scale, the system introduces an extra yaw correction angle θ , representing the global rotation correction around the world up axis, and stores it together with s as the initial calibration result for the session. The ordering is based on a practical observation from early prototypes: the most disruptive calibration errors were often coarse mismatches in perceived distance and orientation, rather than fine-grained geometric distortion. In practice, incorrect scale made virtual movement feel inconsistent with physical motion, while yaw misalignment caused the virtual scene to appear persistently rotated relative to the player’s expected forward direction. For this reason, the system resolves scale and yaw before applying heavier global optimization. This ordering prioritizes the errors that most directly affect player perception and gameplay usability.

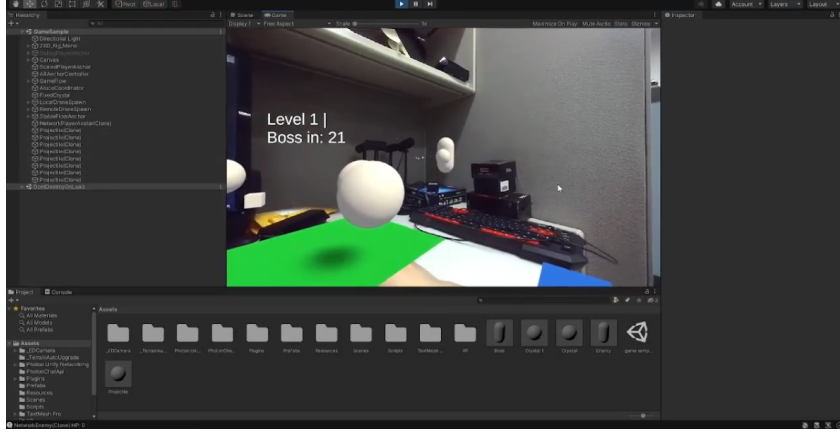


Figure 2: In-game footage.

The estimated scale s does not directly modify the logical coordinate system; instead, it is written to two layers related to visual presentation and movement behavior. First, the `localScale` of the virtual world’s root node is uniformly set to s , so that all gameobjects—enemies, the boss, crystals, the upgrade station, and the landing pads—are scaled by the same factor. Second, the `positionScale` of the player movement magnification node is also multiplied by the same factor, so that the player’s in-game movement distance equals the ZED-measured real displacement multiplied by s . The point of this dual write is that it only affects the visual presentation and the player’s movement amplification, without modifying the underlying logical world coordinates; therefore, a single local calibration error will not propagate into multiplayer synchronization decisions. This arrangement reflects the project’s “fix the logical world; correct the visual presentation locally” layering principle—keeping session-specific corrections inside the presentation layer rather than letting them pollute shared rules [6, 2].

```
// Apply uniform metric correction to the AR world root.
arWorldRoot.localScale = Vector3.one * AppliedContentScale;
arWorldRoot.localRotation = Quaternion.Euler(0f, AppliedYawDegrees, 0f);

// Player movement magnification is rescaled so that
// ZED-measured displacement is mapped consistently into the calibrated game presentation.
scaledPlayerAnchor.positionScale = baseline * contentScaleFactor;
scaledPlayerAnchor.Recenter();
```

2.1.2 Real-World Floor Anchor

Scale consistency alone is not sufficient to define where the virtual content should appear in the physical space. Even after the movement scale is corrected, the system still needs a stable spatial reference that determines the origin and placement of the virtual world relative to the real environment. In this project, the game objects remain fixed in the Unity world, but their physical interpretation depends on how the Unity world is anchored to the ZED-tracked space. The anchor therefore serves as the reference point that connects the logical game world to the player’s real-world environment, allowing enemies, crystals, the boss area, and other gameplay objects to appear around a consistent physical location rather than an arbitrary session-dependent origin. For this reason, the system uses a parent coordinate frame as the anchor layer between the virtual world and the physical space.

The project builds the creation of the real-floor anchor on top of plane raycasting at the screen center [4]. Specifically, the system uses the viewport center as the most natural pointing region for the user, performs a hit test on the detected planes, and only treats a hit plane as

near-horizontal—and therefore valid—if the dot product between its normal and the world up direction exceeds the threshold of 0.85; otherwise it is rejected. The reason is that walls, sloped surfaces, or accidental non-floor geometry should not be misused as the level’s reference plane. To test this function, I build a demo [12] to make sure the detection is stable. After the plane passes validation, the anchor’s orientation does not fully inherit the camera rotation; instead, only the camera’s yaw is taken, deliberately discarding pitch and roll. As a result, the world orientation still follows the player’s horizontal viewing direction, but the level pose will not be tilted simply because the player looks up, looks down, or tilts slightly [3, 12].

Once the anchor is established, the virtual world’s root node is reattached under that anchor and its local transform is reset to the identity. The direct result is that, from this point on, what moves is the camera itself rather than the virtual world’s root node; gameobjects should stably attached to the anchor. The purpose is to stabilize the spatial structure of the scene. The code implementation is shown below:

```
float upDot = Vector3.Dot(plane.worldNormal.normalized, Vector3.up);
if (upDot < minHorizontalDot) return false;

Vector3 forward = cam.transform.forward;
forward.y = 0f;

currentAnchor.position = planePosition;
currentAnchor.rotation = Quaternion.LookRotation(forward.normalized, Vector3.up);
arWorldRoot.SetParent(currentAnchor, false);
```

2.2 Robust Multi-Anchor Similarity Solver

The initial tape-measure calibration can only provide a global scale factor; we must also account for the camera’s own displacement. A single-segment distance cannot simultaneously correct orientation and translation, and more importantly, any single-point sample may be affected by tracking jitter, plane drift, and user operation error. If we write the world reference directly from such a sample, we risk treating accidental noise as a global truth. To address this, the project further introduces a multi-anchor robust similarity transform solver [8], which aims to recover scale, yaw, and translation simultaneously from multiple known correspondences while rejecting outlier samples. In practice, single tape-measure calibration was not sufficient for scenes containing multiple spatially distributed gameplay objects. Although the calibrated scale could make one local distance appear correct, objects farther from the calibration point could still show noticeable position offsets. This was especially visible for objects such as the upgrade station, landing pads, and enemy groups, because their placement depends on the consistency of the entire world reference rather than one measured segment.

Let each anchor sample pair consist of a logical coordinate $\mathbf{p}_k$ and the corresponding local coordinate $\mathbf{q}_k$ reported by ZED. They should satisfy the following similarity transform:

$$\mathbf{q}_k \approx s R_y(\theta) \mathbf{p}_k + \mathbf{t}$$

where $R_y(\theta)$ denotes rotation around the world up axis, s is the unified scale, and $\mathbf{t}$ is the translation. The corresponding least-squares optimization objective can be written as

$$\min_{s, \theta, \mathbf{t}} \sum_k \|\mathbf{q}_k - (s R_y(\theta) \mathbf{p}_k + \mathbf{t})\|^2$$

The reason for restricting the rotation to be around the up axis, rather than allowing fully free 3D rotation, is that at this stage of the project the camera is moved manually as a stand-in

for the drone, and the key interactions take place primarily on the ground reference. Once the gravity direction is sufficiently stable, the most prominent inconsistencies typically come from horizontal orientation, horizontal translation, and overall scale rather than from arbitrary-axis rotation.

In the actual implementation, the samples are first projected onto the XZ plane and centered around their mean. Letting the centered samples remain p_k and q_k , define

$$a = \sum_k (p_k^x q_k^x + p_k^z q_k^z), \quad b = \sum_k (p_k^x q_k^z - p_k^z q_k^x)$$

and obtain the closed-form solution for scale and yaw

$$s = \frac{\sqrt{a^2 + b^2}}{\sum_k \|\mathbf{p}_k\|^2}, \quad \theta = \text{atan2}(b, a)$$

The translation term is then recovered by substituting back through the centroid correspondence. The advantage of this closed form is that it is not only computationally cheap, but also has clear parameter semantics— a and b directly encode the in-plane correlation and rotation bias of the two point sets, making it well suited for repeated solving at runtime [7].

The closed-form solution is not automatically robust by itself. To prevent a single jittery sample from polluting the entire result, the project adopts a RANSAC-style filtering strategy: the system enumerates every pair of two samples as a minimal fitting set, first solves a candidate $(s, \theta, \mathbf{t})$ from these two samples, then uses this candidate to predict all samples and counts as inliers those whose squared residual is below a threshold; after enumerating all sample pairs, it keeps the candidate with the largest inlier count and lowest root mean square error (RSME), and runs the closed-form solver once more on its inlier set to obtain the final result. The core idea here is not to chase complexity, but to acknowledge that real-world samples are not always clean. This is especially true in desk-scale test spaces, where the geometric baseline between anchors is short and any small jitter can easily cause large parameter swings; in the project, after shrinking the anchor baseline to $0.4 \text{ m} \times 0.4 \text{ m}$, this phenomenon became more pronounced. Therefore, generating candidates from the smallest possible sample and re-estimating on the inlier set is a particularly necessary safeguard for small-space experiments [8].

```
// Function context: TryFitFromTwoPairs()
// Two-sample fit on the XZ plane: minimal solver for scale and yaw.
// p1, p2 are the two anchor points in logic space (XZ);
// q1, q2 are their corresponding ZED-reported positions (XZ).
Vector2 dp = p2 - p1;
Vector2 dq = q2 - q1;
scale = Mathf.Max(0.01f, dq.magnitude / dp.magnitude);
float yawRad = Mathf.Atan2(dq.y, dq.x) - Mathf.Atan2(dp.y, dp.x);
yawDeg = yawRad * Mathf.Rad2Deg;
```

```
// Function context: TryFitClosedForm()
// Centroids of logic-space and local-space samples (XZ projection).
Vector2 cp = Vector2.zero, cq = Vector2.zero;
foreach (var d in data) {
    cp += new Vector2(d.logicPoint.x, d.logicPoint.z);
    cq += new Vector2(d.localPoint.x, d.localPoint.z);
}
int n = data.Count;
cp /= n; cq /= n;

// Closed-form similarity fit on inliers (XZ plane).
```



```

float denom = 0f, a = 0f, b = 0f;
for (int i = 0; i < n; i++)
{
    Vector2 p = new Vector2(data[i].logicPoint.x, data[i].logicPoint.z) - cp;
    Vector2 q = new Vector2(data[i].localPoint.x, data[i].localPoint.z) - cq;
    denom += Vector2.Dot(p, p);
    a += p.x * q.x + p.y * q.y;
    b += p.x * q.y - p.y * q.x;
}
float rnorm = Mathf.Sqrt(a * a + b * b);
scale = Mathf.Max(0.01f, rnorm / denom);
float yawRad = Mathf.Atan2(b, a);
yawDeg = yawRad * Mathf.Rad2Deg;

```

```

// RANSAC-style robust selection across all sample pairs.
SolveResult best = default;
for (int i = 0; i < samples.Count; i++)
for (int j = i + 1; j < samples.Count; j++)
{
    if (!TryFitFromTwoPairs(samples[i], samples[j],
                            out float s, out float yaw, out Vector3 t))
        continue;

    var inlierSet = new List<Sample>();
    for (int k = 0; k < samples.Count; k++)
    {
        Vector3 pred = TransformPoint(samples[k].logicPoint, s, yaw, t);
        if ((pred - samples[k].localPoint).sqrMagnitude <= threshold2)
            inlierSet.Add(samples[k]);
    }
    if (inlierSet.Count < 2) continue;

    // Re-fit on the inlier set, then keep the best candidate so far.
    if (!TryFitClosedForm(inlierSet,
                          out float rs, out float ryaw,
                          out Vector3 rt, out float rmse))
        continue;

    bool better = inlierSet.Count > best.inlierCount
        || inlierSet.Count == best.inlierCount && rmse < best.rmse;
    if (better) best = new SolveResult {
        scale = rs,
        yawDeg = ryaw,
        offset = rt,
        rmse = rmse,
        inlierCount = inlierSet.Count
    };
}

```

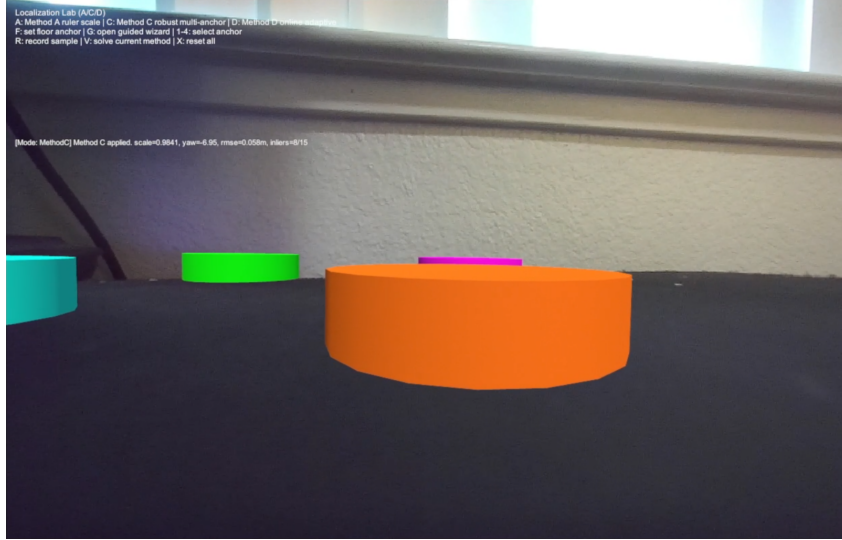


Figure 3: Method run on a $0.4\text{ m} \times 0.4\text{ m}$ desk-scale setup. The HUD reports the solver output: $\text{scale}=0.9841$, $\text{yaw}=-6.95$, $\text{rmse}=0.058\text{ m}$, $\text{inliers}=8/15$.

2.3 Online Adaptive Refinement

Even if the initial calibration is correct, it can only correct a one-shot bias and cannot guarantee that everything stays unchanged throughout the entire run. As the player keeps moving and the scene coverage expands, ZED’s local tracking may drift slowly; if the system stays fixed on its initial parameters, the multiplayer world will gradually accumulate bias. This section is concerned with how to keep the game world stable during long-running tests.

To this end, the system lets samples accumulate during runtime and periodically re-triggers the multi-anchor solver on the accumulated samples at a fixed time interval. The new solution is not hard-cut into the scene; instead, it is continuously linearly blended with the currently active parameters: translation uses Lerp, angles use LerpAngle, and the scale also uses linear interpolation, so that the correction process stays visually smooth rather than letting the player see the entire world snap. The update can be summarized as

$$s_{t+1} = (1 - \alpha) s_t + \alpha \hat{s}$$

$$\mathbf{t}_{t+1} = (1 - \alpha) \mathbf{t}_t + \alpha \hat{\mathbf{t}}$$

$$\theta_{t+1} = \text{LerpAngle}(\theta_t, \hat{\theta}, \alpha)$$

where $(\hat{s}, \hat{\mathbf{t}}, \hat{\theta})$ are the candidate solution newly obtained in the current cycle, and α is a small update step. At the same time, when the RMSE of the current solution is above a threshold, the system actively skips that update, in order to avoid pushing noise directly into the rendered scene [3, 9].

This online correction still acts only on the visual presentation root and does not modify the logical coordinates.

2.4 Multiplayer World Synchronization

In single-player mode, imperfections in the local calibration are usually hard to notice; in two-player mode, however, as soon as both ends calibrate independently and set their own world references, the same upgrade station or the same drone landing pad may end up at different positions on the two devices. Once this discrepancy occurs, subsequent combat hits, item pickups, and spatial interactions all simultaneously lose consistency.

For this reason, the synchronization flow is intentionally designed to be host-authoritative. After both players join the same Photon PUN room, the host first completes its local calibration and fixes three key references: the upgrade station, the host-side landing pad, and the remote-side landing pad. The positions and Euler angles of these references are then packed into a single JSON snapshot. Next, the host uses a PhotonView RPC to broadcast this snapshot once to both the host and the client. After receiving this message, each end then writes these three reference points directly into its own local world and marks the world as locked. The client drone must start from the specific position calibrated by the host, so that the client’s physical position can match the shared world coordinates.

This approach has three implementation features. First, host authority avoids the need to “align two already-different worlds” after both ends calibrate independently; it removes the source of divergence at the architectural level. Second, the JSON snapshot is naturally extensible: it currently carries only the upgrade station and two landing pads, but if more anchor fields need to be added in the future, the overall synchronization paradigm does not have to change. Finally, what the project synchronizes are “key events” and “key references,” not a frame-by-frame copy of the entire world state, so the bandwidth cost is significantly lower than continuous broadcast-style synchronization—for an AR game whose precondition is a shared world, this is a more fitting implementation choice than full-state replication.

```
// 1) Host packs the three key references into a JSON snapshot.
var snap = CalibrationSnapshot.FromTransforms(upgradeStation, localPad, remotePad);
string calibrationJson = snap.ToJson();

// 2) Host broadcasts the snapshot once to all clients (host included).
pv.RPC(nameof(RpcBeginLevel), PhotonTargets.All, calibrationJson);

// 3) Each client deserializes and writes the references into its local world.
[PunRPC]
public void RpcBeginLevel(string calibrationJson) {
    manualCoordinator.ApplyNetworkCalibrationAndStartLevel(calibrationJson);
}
```

2.5 Limitations

Although this project applies several algorithmic and architectural treatments on top of the camera position data provided by the ZED SDK, the experience quality still depends strongly on ZED Camera’s spatial reconstruction. This dependency is more visible in multiplayer mode: since only the host’s world reference is broadcast, the host’s perception of the floor, distance, and orientation largely shapes the spatial quality that both players end up sharing.

In addition, the information that the ZED SDK exposes to Unity is relatively coarse for this use case: developers can stably obtain higher-level results such as poses, planes, and depth, but do not have direct access to finer-grained data such as inlier geometry quality or matching confidence. The scale alignment, robust solving, and online adaptation adopted in this project are therefore best understood as algorithmic mitigations built on top of a limited set of observations. Their effect varies noticeably with environmental factors such as lighting and spatial scale, and

in particular with the size of the test space: in a desk-scale setup the inter-anchor baseline is short, so the robust solver is more sensitive to sample noise, and small local jitter can show up as instability in the solved parameters [5].

3 Setup and Results

After cloning the source code from GitHub, this project can be run directly with Unity on a Windows workstation. The hardware side uses the ZED 2 camera; the software side depends on the ZED SDK and CUDA environment. The ZED plugin version inside Unity is 3.1.0, compatible with ZED SDK 3.7.7 and CUDA 11.7.

The experimental results are mainly demonstrated in the two demo videos below. Video 1 [10] shows the game flow, including initial synchronization, network object consistency, and the basic gameplay loop; the scale alignment in this video uses the two-point distance-ratio approach, taking the ratio between the real camera movement distance and the in-game measured distance as the correction factor. Video 2 [11] shows the multi-anchor spatial alignment test, in which four reference points are used for robust alignment solving—simultaneously correcting scale, yaw, and offset—to validate the system’s spatial consistency and stability in real scenarios.

4 Future Work

Future work proceeds along two routes: one is to turn the currently developed features into reusable tools, and the other is to push the entire system from its current ZED-only form toward the target form facing real drones.

First, the current calibration and multiplayer flows can be further refined into reusable tools—reducing manual debugging costs and helping new members onboard quickly. This work does not change the solution itself, but it can significantly reduce long-term maintenance and extension costs.

Second, on the hardware side, the lab’s previously used RealSense camera can be brought in. RealSense has active depth capability, which complements ZED’s passive vision setup and improves spatial stability at close range and in low-texture regions.

Third, when the target platform truly switches to drones, devices will move faster and cover larger spaces, which will rapidly amplify the errors that are currently acceptable in desk-scale AR tests. Hardware upgrades are therefore not an optional optimization, but a necessary precondition for the platform migration. Once the underlying hardware can stably provide real-scale spatial reconstruction, the “host-authoritative + JSON snapshot broadcast” multiplayer scheme adopted in this project will naturally benefit, because it essentially assumes the existence of a trustworthy shared real-world reference.

5 Conclusion

This project presents a runtime framework for an XR multiplayer drone game using Unity, ZED, and the Photon Plugin. Although the physical drones are not yet fully operational, the framework already brings together spatial input, live calibration, multiplayer synchronization, and a playable demo flow—offering the research team a practical solution that can be further developed.

The results show that using only the ZED Camera as the hardware entry point is sufficient to support the complete runtime process. This includes performing scale alignment and floor anchoring, reducing drift through multi-anchor robust solving and online refinement, and having

the host broadcast a calibration snapshot so both sides start at the same level with a shared world reference. Overall, the project delivers a robust and extensible transitional solution, laying a foundation for future integration with real drone hardware and for larger-scale multiplayer testing.

Acknowledgements

I sincerely thank Professor Nelson Max for his continuous support, thoughtful feedback and guidance throughout this project, which significantly influenced both the technical direction and the overall quality of this work. I also thank my lab colleagues, Prasad Prabhu, Chotrawit Somsri, and Andrew Toreja, for their assistance in the lab, which helped me move the project forward. Finally, I acknowledge that Grammarly and its AI tools were used to improve the writing of this report.

References

- [1] Stereolabs, “Positional Tracking Overview,” official documentation, 2026. Available: <https://www.stereolabs.com/docs/positional-tracking>.
- [2] Stereolabs, “Coordinate Frames,” official documentation, 2026. Available: <https://www.stereolabs.com/docs/positional-tracking/coordinate-frames>.
- [3] Stereolabs, “Positional Tracking Settings,” official documentation, 2026. Available: <https://www.stereolabs.com/docs/positional-tracking/positional-tracking-settings>.
- [4] Stereolabs, “Plane Detection Overview,” official documentation, 2026. Available: <https://www.stereolabs.com/docs/spatial-mapping/plane-detection>.
- [5] Stereolabs, “Camera Calibration,” official documentation, 2026. Available: <https://www.stereolabs.com/docs/video/camera-calibration>.
- [6] Unity Technologies, “Transform.localPosition,” Scripting API, 2026. Available: <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Transform-localhost.html>.
- [7] S. Umeyama, “Least-Squares Estimation of Transformation Parameters Between Two Point Patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991. doi: <https://doi.org/10.1109/34.88573>.
- [8] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981. doi: <https://doi.org/10.1145/358669.358692>.
- [9] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021. doi: <https://doi.org/10.1109/TR0.2021.3075644>.
- [10] “Demo Video 1: Multiplayer Game Flow,” project demonstration video, 2026. Available: https://drive.google.com/file/d/1xxeWGSNKx0s91qBrPC__3XJ4tc_GEtFq/view?usp=drive_link.
- [11] “Demo Video 2: Multi-Anchor Robust Spatial Alignment Test,” project demonstration video, 2026. Available: <https://drive.google.com/file/d/1jNFdSLS08G4SvIFyNn2V92en0bmsLwWD/view?usp=sharing>.

- [12] “Demo Video 3: ZED Plane Detection,” project demonstration video, 2026. Available: <https://drive.google.com/file/d/18aeK2iHkyKFJPNtePuEJAAJN1gW3l3Zw/view?usp=sharing>.